

C++ Practice

Шаблоны

Шаблонные функции

```
int max(int left, int right) {  
    return left > right ? left : right;  
}  
  
int main() {  
    int a = 10, b = 5;  
    std::cout << max(a, b); // 10  
  
    int z = 8.4;  
  
    double c = 7.1, d = 7.3;  
    std::cout << max(c, d); // 7  
}
```

```
int max(int left, int right) {  
    return left > right ? left : right;  
}  
  
double max(double left, double right) {  
    return left > right ? left : right;  
}  
  
int main() {  
    int a = 10, b = 5;  
    std::cout << max(a, b); // 10  
  
    double c = 7.1, d = 7.3;  
    std::cout << max(c, d); // 7.3  
}
```

```
template <typename T>  
T max(T left, T right) {  
    return left > right ? left : right;  
}  
  
int main() {
```

```

    int a = 10, b = 5;
    std::cout << max(a, b); // ?

    double c = 7.1, d = 7.3;
    std::cout << max(c, d); // ?
}

```

Что здесь происходит

Компилятор видит, что мы вызываем функцию `max` 2 раза.

Первый с аргументами типа `int`

Второй с аргументами типа `double`

Посмотрев на эти вызовы, компилятор понимает, что нужно создать две функции, так что код превращается в привычный:

```

template <typename T>
T max(T left, T right) {
    return left > right ? left : right;
}

/* int max(int left, int right) {
    return left > right ? left : right;
}

double max(double left, double right) {
    return left > right ? left : right;
}

*/

int main() {
    int a = 10, b = 5;
    std::cout << max(a, b); // 10

    double c = 7.1, d = 7.8;
    std::cout << max(c, d); // 7.8

    float e = 4.0, z = 5.0;
    std::cout << max(e, z);

}

```

Перегрузка шаблонных функций

Перегрузка функций — это возможность использовать одно и то же имя для нескольких функций, которые отличаются количеством, типами параметров или модификаторами

```
template <typename T>
T max(T left, T right) {
    return left > right ? left : right;
}

/*
double max(double left, double right) {
    return left > right ? left : right;
}
*/

int max(int left, int right) {
    return left > right ? left : right;
}

int main() {
    int a = 10, b = 5;
    std::cout << max(a, b); // 10

    double c = 7.1, d = 7.3;
    std::cout << max(c, d); //
}
```

Пояснение

1. Обычная (нешаблонная) функция всегда имеет приоритет над шаблонной, если они подходят одинаково хорошо.
2. Если можно из шаблонной функции сгенерировать такую функцию, чтобы было меньше приведений типов по сравнению с лучше всего подходящей нешаблонной функцией, то будет сгенерирована именно такая функция.

```
template <typename T, typename U>
void max(T left, U right) {
    std::cout << "<T,U> called" << std::endl;
    std::cout << (left > right ? left : right);
}

/*
Ошибка компиляции, компилятор не знает, какой шаблон использовать
void max(T left, U right) или void max(T left, double right)
Т.к. они равноценны
*/
```

```

template <typename T>
void max(T left, double right) {
    std::cout << "<T,U> called" << std::endl;
    std::cout << (left > right ? left : right);
}
*/

template <typename T>
void max(T left, T right) {
    std::cout << "<T> called" << std::endl;
    std::cout << (left > right ? left : right);
}

/*
void max(int left, int right) {
    std::cout << "<T> called" << std::endl;
    std::cout << (left > right ? left : right);
}
*/

int main() {
    int a = 10, b = 5;
    max(a, b); // 10

    double c = 7.1, d = 7.3;
    max(c, d); // 7

    max(a, c);
}

```

3. Если подходит несколько шаблонных функций, то из них выбирается более специализированная.

Можно явно указывать, какую функцию мы хотим вызвать:

```

template <typename T, typename U>
void max(T left, U right) {
    std::cout << "<T,U> called" << std::endl;
    std::cout << (left > right ? left : right);
}

template <typename T>
void max(T left, T right) {
    std::cout << "<T> called" << std::endl;
    std::cout << (left > right ? left : right);
}

```

```

}

int main() {
    int a = 10, b = 5;
    max<int, double>(a, b);
}

```

Шаблонные классы

```

template <typename T>
class Storage {
    T _value;
public:
    Storage(T value) : _value(value) { }

    void printValue() {
        std::cout << _value << std::endl;
    }
};

int main() {
    Storage<int> a(5);
    Storage<double> b(10.0);

    a = b; // нельзя
}

```

Разделить объявление и реализацию шаблонного класса на два файла `.h` и `.cpp` классическим способом не получится. Есть разные подходы, но чаще всего всю реализацию пишут в одном файле с объявлением `Storage.h`

```

template <typename T>
class Storage {
    T _value;
public:
    Storage(T value);

    void printValue();
};

template <typename T>
Storage<T>::Storage(T value) : _value(value)
{ }

```

```

template <typename T>
void Storage<T>::printValue()
{
    std::cout << _value << std::endl;
}

```

main.cpp

```

#include "Storage.h"

int main() {
    Storage<int> a(5);
    a.printValue();
}

```

Шаблонные методы внутри шаблонного класса

```

template <typename T>
class Storage {
    T _value;
public:
    Storage(T value);

    void printValue();

    template <typename U>
    void printSomeValue(U someValue);
};

template <typename T>
Storage<T>::Storage(T value) : _value(value)
{ }

template <typename T>
void Storage<T>::printValue()
{
    std::cout << _value << std::endl;
}

template <typename T>
template <typename U>
void Storage<T>::printSomeValue(U someValue)
{

```

```
    std::cout << someValue << std::endl;
}
```

main.cpp

```
#include "Storage.h"

int main() {
    Storage<int> a(5);
    a.printValue(); // 5

    a.printSomeValue(10.0); // 10.0
}
```

Специализация шаблонов

Мы можем создать отдельную реализацию класса/структуры для конкретных типов

```
#include <iostream>
#include <iomanip>

template <typename T>
struct ValueFormatter {
    T _value;

    ValueFormatter(T value) : _value(value) {}

    void printFormatted() {
        std::cout << _value << std::endl;
    }
};

int main() {
    ValueFormatter<int> counter(5);
    counter.printFormatted();      // 5

    ValueFormatter<double> counter(10.0);
    counter.printFormatted();      // 10.0
}
```

```
#include <iostream>
#include <iomanip>

template <typename T>
```

```

struct ValueFormatter {
    T _value;

    ValueFormatter(T value) : _value(value) {}

    void printFormatted() {
        std::cout << _value << std::endl;
    }
};

template <>
struct ValueFormatter<double> {
    double _value;

    ValueFormatter(double value) : _value(value) {}

    void printFormatted() {
        std::cout << std::fixed << std::setprecision(2) << _value <<
std::endl;
    }
};

int main() {
    ValueFormatter<int> counter(5);
    counter.printFormatted();          // 5

    ValueFormatter<double> price(10.0);
    price.printFormatted();            // 10.00
}

```

Частичная специализация

Частичная специализация — это отдельная версия шаблона не для одного конкретного типа, а для целой группы похожих типов.

```

template <typename T>
struct TypeInfo {
    static void print() {
        std::cout << "Ordinary type" << std::endl;
    }
};

int main() {
    TypeInfo<int>::print();             // Ordinary type // T == int
    TypeInfo<double>::print();          // Ordinary type // T == double
}

```



```
TypeInfo<double*>::print();    // Ordinary type // T == double*
}
```

Теперь представим, что мы хотим добавить отдельное поведение для указателей. Например, чтобы `int*`, `double*`, `char*` обрабатывались не как обычные типы, а как указатели.

```
#include <iostream>

template <typename T>
struct TypeInfo {
    void print() {
        std::cout << "Ordinary type" << std::endl;
    }
};

// Частичная специализация для всех указателей
template <typename T>
struct TypeInfo<T*> {
    static void print() {
        std::cout << "Pointer type" << std::endl;
    }
};

int main() {
    TypeInfo<int> object;

    object.print();

    TypeInfo<int>::print();    // Ordinary type // T == int
    TypeInfo<double>::print(); // Ordinary type // T == double

    TypeInfo<int*>::print();   // Pointer type // T == int
    TypeInfo<double*>::print(); // Pointer type // T == double
}
```

Специализация шаблонных функций

```
#include <iostream>

template <typename T>
void printValue(T value) {
    std::cout << value << std::endl;
}
```

```

// Полная специализация для double
template <>
void printValue<double>(double value) {
    std::cout << "Double value: " << value << std::endl;
}

// Перегрузка для double
/* void printValue(double value) {
    std::cout << "Double value: " << value << std::endl;
} */

int main() {
    printValue(10);          // 10
    printValue(3.14);        // Double value: 3.14
    printValue("hello");     // hello
}

```

На практике для функций чаще используют перегрузку, потому что она проще читается и лучше вписывается в обычный механизм выбора функций компилятором.

Нетиповые параметры шаблонов

```

template <typename T, int N>
struct Array {
    T data[N];
};

// std::array<int, 5> myArr

int main() {
    Array<double, 5> myArr;

    for (int i = 0; i < 5; ++i) {
        myArr.data[i] = 7;
    }

    // myArr.data[7] //
}

```

Значения нетиповых параметров шаблонов должно быть известно в момент компиляции

```

template <typename T, int N>
struct Array {

```

```

    T data[N];
};

/*
struct Array {
    double data[5];
};
*/

int main() {
    Array<double, 5> myArr;

    for (int i = 0; i < 5; ++i) {
        myArr.data[i] = 7;
    }
}

```

```

template <typename T, int N = 5>
struct Array {
    T data[N];
};

/*
struct Array {
    double data[???];
};
*/

int main() {
    int n;
    std::cin >> n;
    Array<double, n> myArr;

    for (int i = 0; i < 5; ++i) {
        myArr.data[i] = 7;
    }
}

```

```

template <typename T, int N>
struct Array {
    T data[N];
};

/*

```

```

struct Array {
    double data[5];
};
*/

int main() {
    const int size = 5; // ok

    Array<double, size> myArr; // ok

    for (int i = 0; i < size; ++i) {
        myArr.data[i] = 7;
    }
}

```

```

template <typename T, int N>
struct Array {
    T data[N];
};

/*
struct Array {
    double data[5];
};
*/

int main() {
    int n;
    std::cin >> n;
    // const int size = 7; // ok
    constexpr int size = n; // не сработает, ошибка компиляции

    Array<double, size> myArr;

    for (int i = 0; i < size; ++i) {
        myArr.data[i] = 7;
    }
}

```

Вычисления во время компиляции

```

#include <iostream>

int factorial(int n) {

```

```

        if (n == 1)
            return 1;

        return n * factorial(n - 1);
    }

int main() {
    // static_assert(factorial(5) == 120); нельзя

    int size = factorial(4);
    int numbers[size]; // проблема

    std::cout << "factorial<4>() = " << size << std::endl;
    std::cout << "array size = " << sizeof(numbers) / sizeof(numbers[0]) <<
std::endl;
}

```

```

#include <iostream>

template <int N>
constexpr int factorial() {
    return N * factorial<N - 1>();
}

/*
constexpr int factorial() {
    return 4 * factorial<3>();
}

constexpr int factorial() {
    return 3 * factorial<2>();
}

constexpr int factorial() {
    return 2 * 1;
}
*/

// Полная специализация для N = 1
template <>
constexpr int factorial<1>() {
    return 1;
}

int main() {

```

```

static_assert(factorial<5>() == 120); // проверка во время компиляции

constexpr int size = factorial<4>(); // 24
constexpr int size = 24;
int numbers[size];

std::cout << "factorial<4>() = " << size << std::endl;
std::cout << "array size = " << sizeof(numbers) / sizeof(numbers[0]) <<
std::endl;
}

```

Классы и структуры в виде параметров шаблонов

```

struct Date {
    int day, month, year;
};

class User {
    std::string name;
    Date birthday;
public:
    User(std::string name, Date birthday)
        : name(name), birthday(birthday) { }

    std::string getName() const {
        return name;
    }
};

class Pet {
    std::string name;
    Date birthday;
public:
    Pet(std::string name, Date birthday)
        : name(name), birthday(birthday) { }

    std::string getName() const {
        return name;
    }
};

template <class T>
void printName(T& namedObject) {
    std::cout << namedObject.getName();
}

```

```
int main() {
    User user("Ivan", {24, 12, 2003});
    Pet pet("Cat", {17, 3, 2021});

    printName(user);
    printName(pet);
}
```

Задание

- ☐ Реализовать шаблонную структуру `Pair<T1, T2>`
 - ☐ Возможность доступа к содержимому (`pair.first = 1, pair.second = 4.5`)
- ☐ Реализовать класс `Array<T, N>`
 - ☐ все конструкторы, операторы присваивания, копирование + перемещение
 - ☐ поддержка `std::initializer_list`
 - ☐ методы доступа
 - ☐ `at()`, `operator[]`, `front()`, `back()`, `data()`
 - ☐ размер
 - ☐ `bool empty()`, `size()`, `max_size()`
 - ☐ оператор вывода в консоль
 - ☐ `std::cout << myArr << std::endl;`
- ☐ Реализовать класс `Vector<T>` (динамический размер + реаллокация)
 - ☐ все конструкторы, операторы присваивания, копирование + перемещение
 - ☐ поддержка `std::initializer_list`
 - ☐ методы доступа
 - ☐ `at()`, `operator[]`, `front()`, `back()`, `data()`
 - ☐ размер
 - ☐ `bool empty()`, `size()`, `max_size()`, `reserve()`, `capacity()`
 - ☐ оператор вывода в консоль
 - ☐ `std::cout << myArr << std::endl;`
 - ☐ методы модификации
 - ☐ `clear()`, `insert(index, value)`, `erase(index)`, `push_back(value)`, `resize()`
- ☐ Реализовать шаблонную функцию `sumValues()`, возвращающую сумму элементов созданных классов
 - ☐ Добавить перегрузку этой функции для случаев, когда массивы хранят указатели

```
int a = 10;
int b = 20;
int c = 30;

Vector<int*> vectorValues = { &a, &b, &c };
Array<int*, 3> arrayValues = { &a, &b, &c };
std::vector<int> values = {10, 7, 5};

std::cout << sumValues(vectorValues) << std::endl; // 60
std::cout << sumValues(arrayValues) << std::endl; // 60
std::cout << sumValues(values) << std::endl; // 22
```

- ☐ Реализовать шаблонный класс `NumericStorage<ContainerType>`, который хранит ваш массив
- ☐ `ContainerType` может быть `Vector<T>` или `Array<T, N>`
 - ☐ умеет печатать все элементы
 - ☐ умеет считать сумму всех элементов
 - ☐ имеет шаблонный метод `multiplyBy<Factor>(Factor factor)`, который умножает каждый элемент на значение другого типа